Dossier de Conception Applicative

Application Weeb

Version: 1.0

Périmètre: Frontend React + Backend Django REST

Auteur: Théodore Abitbol

Date: 2026-02-27

Sommaire

1. Prise en main du code

2. Decomposition et fonctionnement global

3. Structure du projet Django

4. Structure de la base de données

5. Règles de gestion: authentification et rôles

6. Diagramme de séquence: authentification

7. Recommandation d'environnement informatique

8. Patterns logiciels et classes

9. Plan d'amélioration priorisé

1. Prise en main du code

1.1 Cartographie rapide

L'application **Weeb** repose sur une séparation claire entre deux sous-systèmes complémentaires. Le dossier `backend/` porte les responsabilités serveur, c'est-a-dire l'exposition de l'API REST, la gestion de l'authentification, la logique utilisateur et le cycle de vie des articles. Le dossier `frontend/` porte la responsabilité de l'expérience utilisateur, avec une application React monolithique qui gère la navigation, les formulaires et la consommation de l'API.

Cette organisation est renforcée par une découpe métier interne lisible. Côté backend, `config/` centralise la configuration globale (paramétrage, routage principal), `users/` concentre les cas d'usage liés aux comptes et `blog/` couvre la publication de contenu. Côté frontend, les composants transverses sont placés dans `src/components`, les pages métier dans `src/pages`, et la couche d'accès HTTP est factorisée dans `src/api/axios.js`. Cette cartographie permet d'identifier rapidement ou lire et ou modifier selon la nature d'un besoin.

1.2 Parcours de lecture recommandé

La prise en main est plus efficace lorsqu'elle suit une progression du général vers le spécifique. Il est pertinent de commencer par `backend/config/settings.py` afin de comprendre le cadre d'exécution, notamment l'authentification, CORS/CSRF, les choix JWT et la base de données. Cette première étape donne le contexte technique indispensable pour interpréter correctement le reste du code.

La lecture peut ensuite se poursuivre dans `backend/config/urls.py`, qui fournit une vue contractuelle des points d'entrée API. Une fois ce cadre posé,

l'analyse des applications `users/` puis `blog/` permet de comprendre les règles métier concrètement implémentées. Enfin, le passage au frontend via `frontend/src/api/axios.js`, `App.jsx` et `Nav.jsx` permet de relier la logique client au comportement backend et d'observer comment la session utilisateur est maintenue dans l'interface.

1.3 Constats de prise en main

Les responsabilités principales sont isolées et la séparation `users`/`blog` limite l'entrelacement de logique métier. L'authentification est cohérente avec les bonnes pratiques : l'access token reste en mémoire et le refresh token est stocké en cookie HttpOnly, ce qui réduit l'exposition aux attaques de type vol de token via JavaScript.

Le principal point de vigilance se situe sur la fiabilisation. Les fichiers de tests existent mais ne contiennent pas de couverture effective, ce qui augmente le risque de régression silencieuse lorsque des modifications sont apportées au flux d'authentification, aux permissions ou au cycle de publication.

2. Décomposition et fonctionnement global

2.1 Vue d'ensemble

Le fonctionnement global suit un modèle client-serveur classique: la SPA React interagit avec l'API Django au travers d'Axios, avec transmission des cookies activée pour permettre le mécanisme de refresh de session. Les parcours publics, comme la consultation du blog et la lecture d'un article, sont accessibles sans authentification, tandis que les parcours d'écriture, comme l'ajout et l'édition d'un article, nécessitent une session valide.

D'un point de vue métier, l'application enchaîne six processus principaux qui structurent l'expérience utilisateur. Le premier est l'inscription, qui crée un compte initialement inactif. Le deuxième est la connexion, qui initialise la session JWT. Le troisième concerne la consultation libre du contenu. Le quatrième couvre la publication par un utilisateur authentifié. Le cinquième encadre la modification par le propriétaire du contenu. Le sixième gère la réinitialisation de mot de passe via email et token temporaire. Cette séquence de processus forme une chaîne logique complète entre création de compte, accès et production de contenu.

2.2 Contraintes, vulnérabilités et priorisation

Si l’ensemble va dans le bon sens en termes de bonnes pratiques, en particulier la courte durée de vie des tokens et l'usage d'un cookie HttpOnly pour le refresh, ces choix ne suffisent pas à eux seuls pour un niveau de robustesse de production. L'absence de limitation de tentative de connexion, la configuration de développement encore active et le manque de tests automatisés constituent les risques les plus critiques. Avec une équipe plus conséquente, il aurait été possible de pallier ces problèmes sans difficultés.

La priorisation retenue place en premier l'authentification et la gouvernance des accès, car ce sont les zones les plus sensibles en termes de sécurité et d'impact métier. Vient ensuite la fiabilité opérationnelle, avec la migration de l'email de console vers un service réel et l'évolution de la base SQLite vers PostgreSQL pour supporter mieux la concurrence. Les optimisations de confort, comme la restauration de session au rechargement, sont importantes mais passent après les sujets de sécurité et de stabilité.

3. Structure du projet Django

3.1 Configuration globale (`config`)

Le module `config` joue un rôle de colonne vertébrale du backend. Le fichier `settings.py` formalise les choix structurants du projet, comme l'utilisation d'un modèle utilisateur personnalisé, l'activation de Django REST Framework, la stratégie JWT et les règles CORS/CSRF pour l'environnement local. Ce fichier conditionne directement les comportements transverses de sécurité et d'intégration entre frontend et backend.

Le fichier `urls.py` traduit ensuite cette configuration en surface API exploitable. Les routes d'authentification sont exposées au niveau global, puis délèguent les parcours métier aux applications `users` et `blog`. Cette composition rend la lecture des points d'entrée très rapide et facilite l'évolution du contrat API sans multiplier les couplages.

3.2 Application `users`

L'application `users` concentre la logique d'identité et de session. Le modèle `User`, basé sur `AbstractUser`, impose l'unicité de l'email et fait de celui-ci

l'identifiant de connexion, ce qui aligne mieux le comportement technique avec l'usage attendu par les utilisateurs finaux.

Les serializers structurent la validation et la représentation des données pour chaque cas d'usage: inscription, consultation du profil courant et réinitialisation de mot de passe (implémentée de façon superficielle ici, sans SMTP). Cette centralisation de validation évite d'éparpiller les règles dans les vues. Les vues, de leur côté, orchestrent les opérations: création de compte, lecture du profil `me`, émission des tokens, rafraîchissement de session et traitement du reset mot de passe. L'ensemble forme un module cohérent, dont le cycle fonctionnel couvre toutes les étapes de la vie d'un compte.

3.3 Application `blog`

L'application `blog` porte la logique éditoriale. Le modèle `Post` associe chaque article à un auteur et conserve les métadonnées temporelles qui permettent le tri et la traçabilité. Le serializer enrichit la représentation de l'auteur pour faciliter l'affichage côté frontend sans nécessiter de requêtes supplémentaires.

Le `PostViewSet` offre un CRUD complet, complété par une action d'édition explicite utilisée par l'interface. Les permissions s'appuient sur une règle simple et robuste: la lecture reste ouverte, mais toute modification est réservée au propriétaire. Cette combinaison entre ViewSet, serializer et permission produit une API concise tout en conservant un bon niveau de contrôle.

3.4 Architecture logique backend

L'architecture backend suit une logique en couches qui favorise la maintenabilité. La couche présentation gère les requêtes HTTP et le routage

applicatif. La couche serializer prend en charge la validation et la transformation de données. La couche modèle encapsule la persistance et les relations métier. Enfin, la couche permission porte les politiques d'accès.

Cette séparation des rôles permet de localiser rapidement une anomalie selon sa nature, de réduire les effets de bord lors des changements et de rendre les tests plus simples à cibler lorsque la couverture sera introduite.

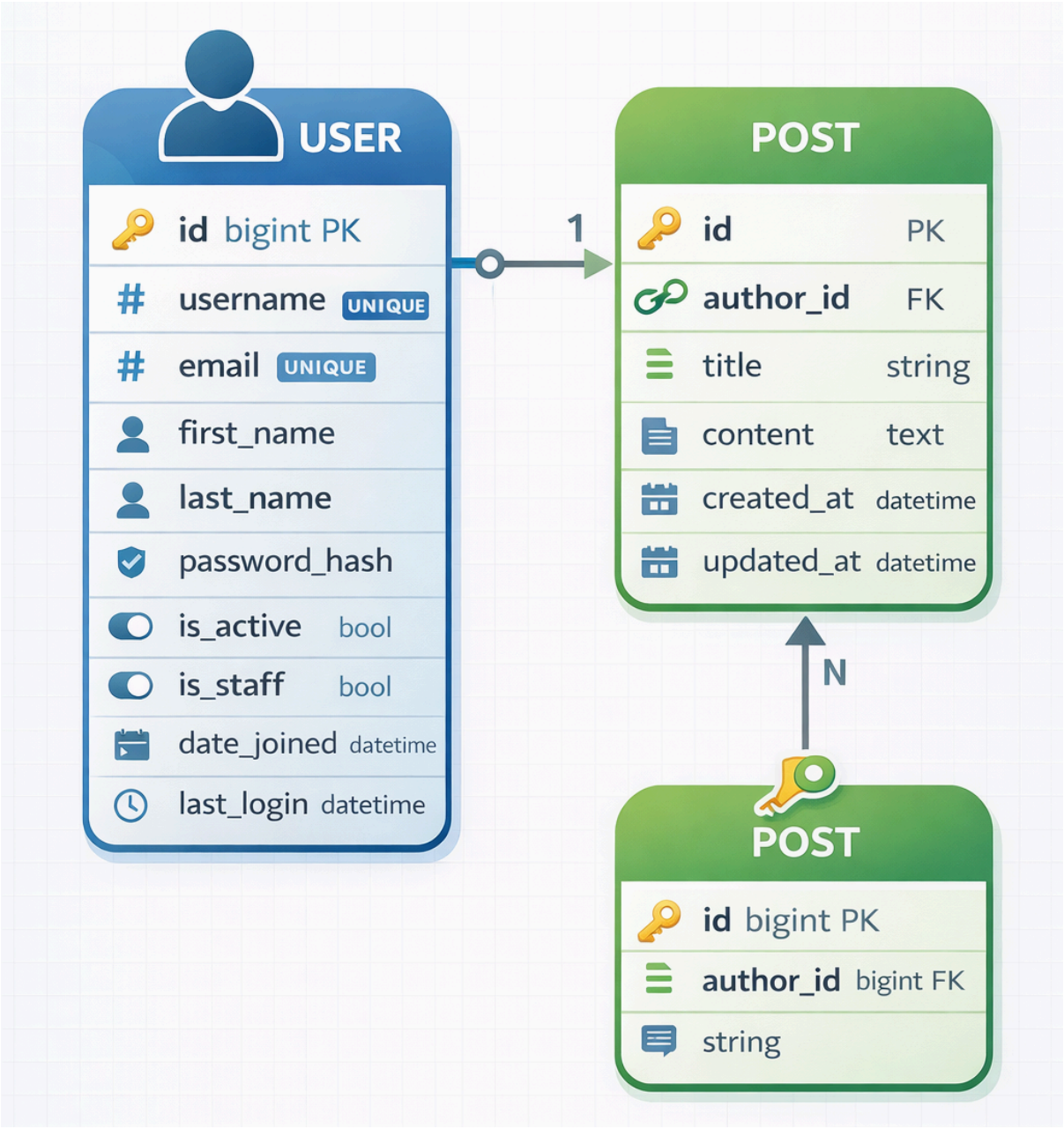
4. Structure de la base de données

4.1 Entités et relations

Le schéma de données est volontairement minimaliste et aligné sur le besoin fonctionnel actuel. Deux entités suffisent à représenter le cœur métier : l'utilisateur, qui porte l'identité et les états de compte, et l'article, qui porte le contenu publié. La relation un-a-plusieurs entre utilisateur et article établit clairement la propriété du contenu.

Plusieurs choix de conception renforcent la cohérence du modèle. L'email unique garantit l'absence d'ambiguïté lors de l'authentification. La suppression en cascade sur l'auteur protège l'intégrité référentielle. Les horodatages automatiques facilitent l'ordre d'affichage et les besoins de diagnostic fonctionnel sans complexité supplémentaire.

4.2 Diagramme conceptuel (ER)



4.3 Evolutions BDD recommandées

Les évolutions prioritaires de la base doivent accompagner la montée en charge et la gouvernance métier. L'ajout d'un index sur `created_at` est une optimisation directe pour la consultation chronologique du blog. La formalisation des rôles, via les mécanismes natifs Django ou un modèle

dédié, permettra d'exprimer clairement les habilitations à mesure que l'organisation fonctionnelle se complexifie.

A moyen terme, une table d'audit des actions sensibles serait pertinente pour renforcer la traçabilité et soutenir les exigences de sécurité opérationnelle. Cette évolution doit être pensée avec une politique de rétention pour conserver un bon compromis entre valeur d'observabilité et coût de stockage. Mais cela, encore une fois, dépasse largement la faisabilité et les coûts engagés d’un blog de petite taille avec une équipe réduite.

5. Règles de gestion: authentification et rôles

5.1 Règles actuellement implémentées

Le cycle d'authentification suit une logique explicite en plusieurs étapes. L'inscription crée un compte inactif, ce qui introduit un sas de validation avant accès effectif. La connexion retourne un accès token pour les appels API immédiats, tandis que le refresh token est isolé dans un cookie HttpOnly pour limiter l'exposition côté navigateur.

En exécution courante, le frontend transmet l'access token dans l'en-tête `Authorization`. En cas d'expiration, une tentative de rafraîchissement est faite automatiquement. Si elle échoue, la session est purgée côté client. La déconnexion supprime explicitement le cookie de refresh. Sur le domaine blog, les règles sont cohérentes avec la responsabilité éditoriale: lecture publique, écriture réservée aux authentifiés, et modification strictement réservée au propriétaire de l'article.

5.2 Rôles métier identifiés

Le code fait apparaitre quatre niveaux d'acteurs. Le visiteur anonyme peut consulter le contenu sans contribuer. L'utilisateur authentifié peut publier. L'auteur, en tant que propriétaire, peut modifier ses propres articles. Enfin, le staff ou superuser dispose d'un rôle d'administrateur technique via l'interface Django Admin.

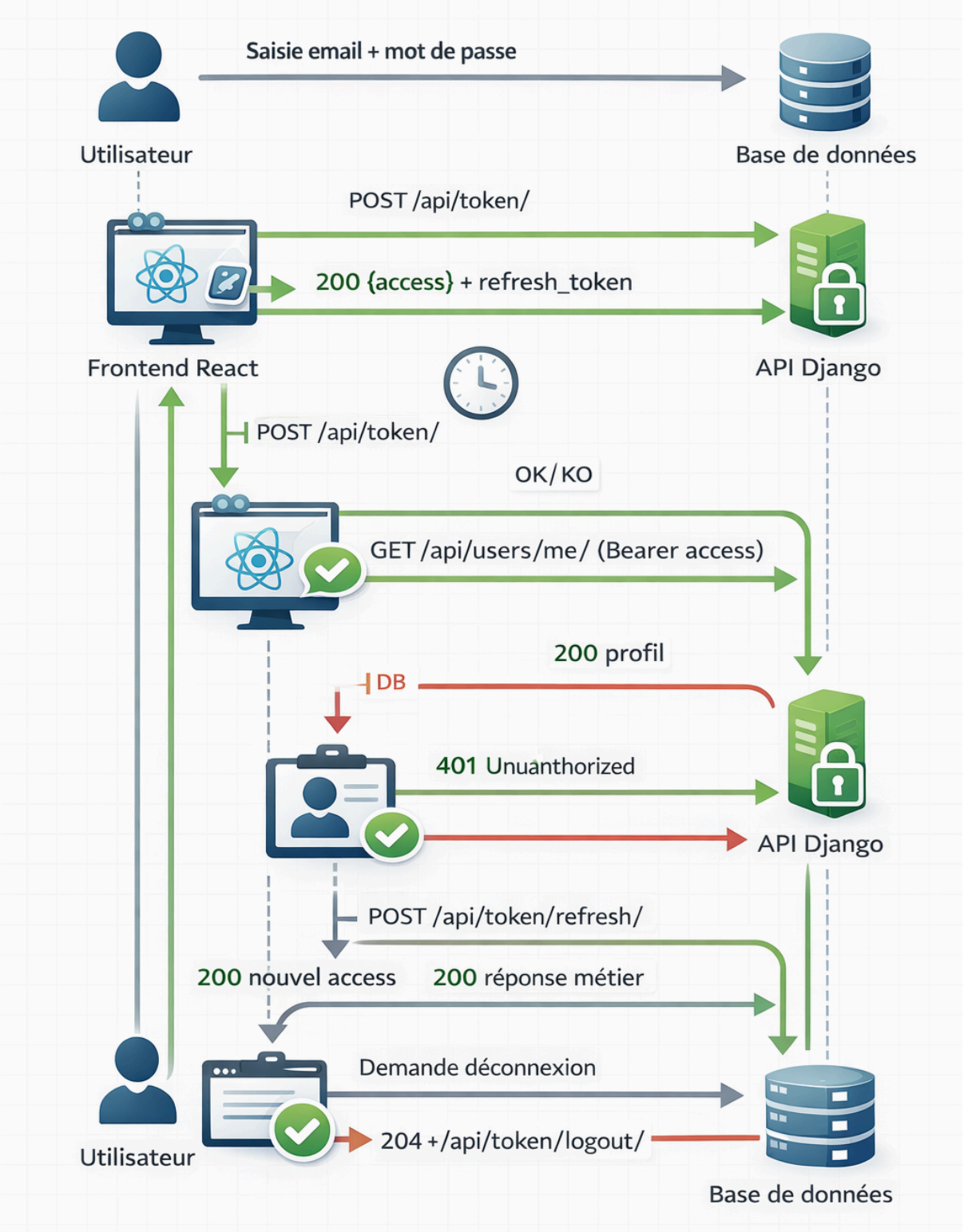
La limite actuelle est l'absence de rôle métier explicite pour la validation fonctionnelle des comptes et, potentiellement, la modération de contenu. Cette responsabilité existe de fait dans le processus, mais elle n'est pas encore modélisée formellement dans le système d'habilitation.

5.3 Matrice d'accès

La matrice d'accès ci-dessous formalise les droits observés dans l'état actuel du code. Elle met en évidence que le principal critère d'autorisation est l'authentification, complète par la condition de propriété pour les opérations d'édition et de suppression.

6. Diagramme de séquence: authentification

Le diagramme suivant décrit le chemin nominal de connexion, puis le comportement de rattrapage en cas d'expiration du token d'accès. Il met en lumière la coopération entre frontend, API et base de données, ainsi que le rôle du cookie de refresh dans la continuité de session.



7. Recommandation d'environnement informatique

7.1 Cible recommandée

Le déploiement peut être réalisé simplement avec Vercel pour le frontend React et Render pour le backend Django. Vercel héberge l’application web statique, tandis que Render exécute l’API Django avec Gunicorn. Cette approche évite une infrastructure trop complexe tout en restant adaptée à un projet fullstack moderne.

La base de données doit évoluer vers PostgreSQL pour gagner en robustesse transactionnelle et en capacité concurrente. En complément, un service Redis peut soutenir les usages transverses comme le rate-limiting ou des évolutions asynchrones. Enfin, un pipeline CI/CD outille avec lint, tests et contrôles sécurité permet de fiabiliser les livraisons, bien que cela soit, à mon sens, over-engineered pour un simple blog.

7.2 Interactions entre composants

Le flux d'exécution suit une chaîne simple et maîtrisable. Le navigateur charge d'abord les ressources statiques de l'application cliente. La SPA envoie ensuite ses requêtes métier vers l'API Django en HTTPS. Le backend persiste les données dans PostgreSQL et s'appuie sur un service email externe pour les notifications sensibles, comme la réinitialisation de mot de passe.

Ce socle doit être complété par une couche d'observabilité centralisée, afin de suivre les métriques de performance, les erreurs applicatives et les événements de sécurité. Cette visibilité est essentielle pour diagnostiquer rapidement et piloter l'amélioration continue.

7.3 Choix techniques comparées

Le passage de l'état actuel a la cible recommandée repose sur un arbitrage pragmatique entre simplicité, fiabilité et évolutivité. SQLite et le serveur Vite conviennent au développement, mais deviennent limitants pour une exploitation durable. PostgreSQL, un delivery frontend statique et un service email de production offrent une meilleure tenue en charge et une meilleure maîtrise opérationnelle.

7.4 Réduction de l'impact écologique

La réduction de l'impact écologique doit être intégré comme un critère de conception, et non ajoutée a posteriori. La première mesure consiste à limiter le surprovisionnement grâce à un autoscaling borne et à une extinction automatique des environnements non productifs hors plages utiles. La seconde vise l'efficacité matérielle, notamment via des instances ARM lorsque disponibles.

Sur le plan applicatif, l'optimisation des transferts est déterminante: cache HTTP, payloads API plus fins et pagination systématique réduisent la consommation réseau et calcul. Le suivi de KPI GreenOps (usage CPU, trafic sortant, taux de cache) permet ensuite d'objectiver les gains et d'orienter les optimisations futures.

8. Patterns logiciels et classes

8.1 Patterns déjà utilisés

Le projet applique déjà plusieurs patterns utiles qui expliquent sa visibilité actuelle. Le style MTV de Django structure proprement la séparation des responsabilités. Le pattern Serializer de DRF centralise les validations et rend le contrat API explicite. La classe de permission `IsAuthorOrReadOnly` incarne une politique d'accès claire et réutilisable.

Côté frontend, l'interceptor Axios joue un rôle transversal en gérant de manière uniforme les tokens et le rafraîchissement de session. La protection de routes via `ProtectedRoute` complete cette approche en contrôlant l'accès aux écrans sensibles au niveau de la navigation. L'ensemble forme une architecture modulaire, facilement extensible si les mêmes principes sont conservés.

8.2 Patterns recommendés

Pour accompagner l’évolution du projet, on peut ajouter une couche Service dans le backend.
Concrètement, cela permet de mettre la logique métier (règles de création, validation, autorisations) hors des vues HTTP, ce qui rend le code plus lisible et plus facile à tester.
Il est aussi utile de formaliser les droits avec un système de rôles (RBAC), plutôt que de se limiter à la règle “auteur ou non”.
Par exemple, distinguer clairement les permissions d’un admin, d’un auteur et d’un utilisateur standard.
Si les accès base de données deviennent plus complexes, on pourra ensuite introduire un pattern Repository pour regrouper les requêtes avancées dans des classes dédiées.
Ce n’est pas indispensable immédiatement, mais cela peut aider à garder une architecture maintenable à moyen terme.

9. Plan d'amélioration priorisé

La feuille de route court terme doit d'abord viser la réduction du risque. La priorité absolue est l'ajout de tests API couvrant l'authentification, le refresh token, les permissions d'auteur et la réinitialisation de mot de passe. Sans

cette base, toute évolution fonctionnelle augmente la probabilité de régression.

La deuxième priorité est la formalisation du RBAC afin de rendre explicites les rôles de validation et d'administration métier.

La troisième est le durcissement de la configuration de production, en particulier les variables de sécurité exposées par défaut en développement.

Enfin, la migration vers PostgreSQL et l'ajout d'outils de monitoring/alerting complèteront la stabilisation technique et opérationnelle de la plateforme.

Conclusion

L'application **Weeb** dispose déjà d'une architecture cohérente pour un socle MVP full stack, avec une séparation claire des responsabilités et un flux d'authentification globalement sain. Le chantier principal n'est pas une refonte structurelle, mais une consolidation méthodique autour de trois axes: fiabilité par les tests, gouvernance des accès par les rôles, et robustesse d'exploitation par le durcissement de l'environnement cible.